



Experiment 8

Student Name: Analava Bera
Branch: CSE
Semester: 6th
Subject Name: System Design

UID: 23BCS13611
Section/Group: KRG 3-A
Date of Performance: 10/04/2026
Subject Code: 23CSH-314

1. Aim : To design a stock trading platform (similar to Zerodha, Groww & Upstox) that enables users to trade stocks, view real-time market data, manage portfolios, and execute buy/sell orders with high consistency and low latency..

2. Objective:

- To design a scalable architecture for a stock trading platform.
- To enable real-time stock price updates using WebSockets.
- To support order placement, modification, and cancellation.
- To ensure high consistency and correctness in financial transactions.
- To implement portfolio tracking and watchlist features.

3. Tools Used:

- Frontend (ReactJS, HTML, CSS, JavaScript, WebSocket) o Built responsive UI with real-time stock updates
- Backend (Node.js / Java / Spring Boot) o Developed REST APIs for trading, authentication, and portfolio
- Database (PostgreSQL / MySQL) o Managed financial transactions and user data
- Redis o Used for caching stock prices and improving latency
- Message Systems / WebSocket o Enabled real-time updates for stock prices and order status
- Deployment (AWS / GCP, Load Balancer, Docker) o Scalable cloud infrastructure with containerization

4. System Requirements:

A. Functional Requirements

- Users should be able to register, login, and complete KYC verification.

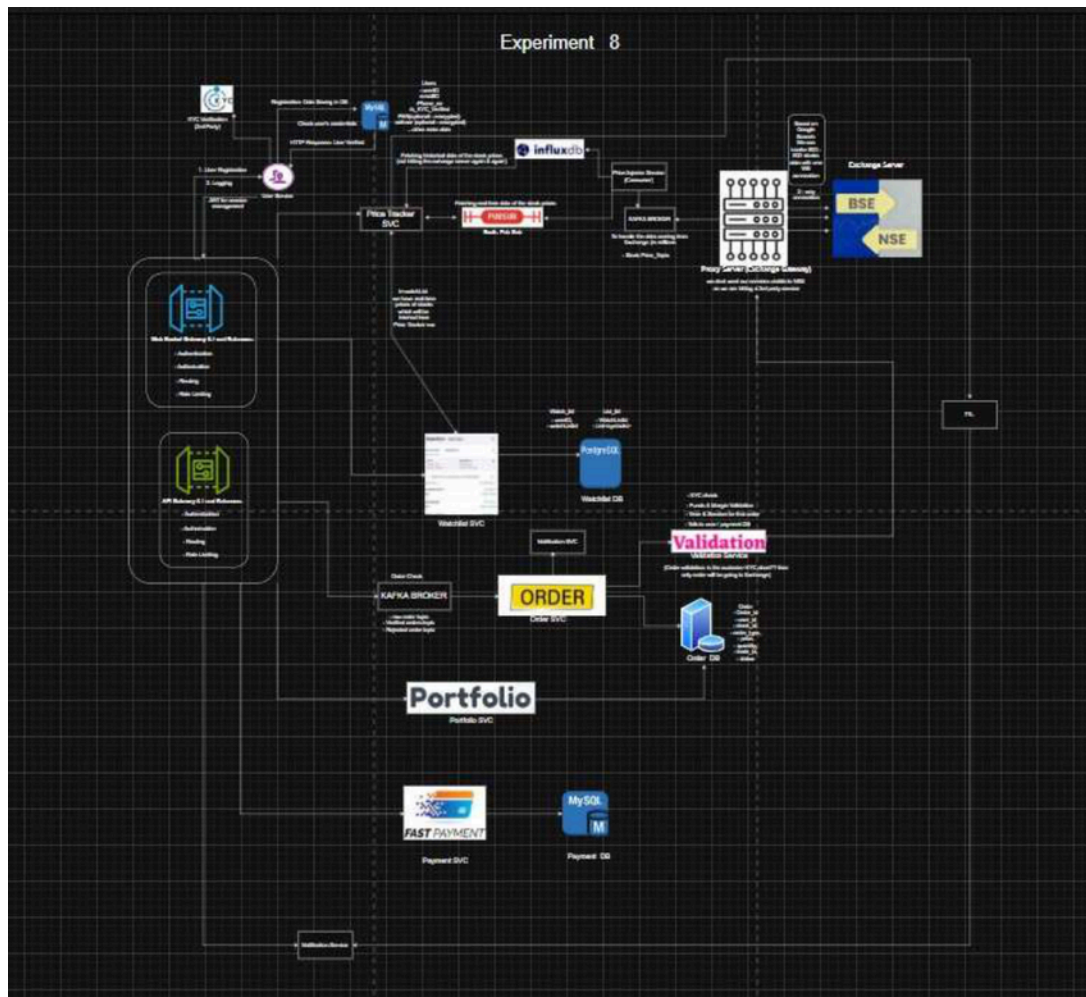


- Users should be able to view real-time stock prices and historical data.
- Users should be able to place, modify, and cancel buy/sell orders.
- The system should support limit orders and market orders.
- Users should be able to maintain a watchlist with real-time updates.
- Users should be able to view portfolio, holdings, and performance.
- The system should provide order status updates in real-time.
- Users should be able to deposit and withdraw funds.

B. Non-Functional Requirements

- Scalability
System should support 2–3 million daily active users with high-frequency trading and thousands of stocks.
- CAP Theorem
Consistency >>> Availability
Financial correctness is critical (no duplicate trades, no wrong balances).
- Latency:
Order placement latency should be < 100 ms
Stock price updates latency should be < 50 ms
- Reliability:
System must ensure fault tolerance and no data loss in transactions.

5. Low Level Design (LLD):



6. Scalability Solution

- Use WebSockets for real-time stock price updates
- Implement Redis caching for frequently accessed data
- Use load balancers for distributing traffic
- Use microservices architecture (Order, Portfolio, Market Data, Funds)
- Store transactional data in strongly consistent databases
- Use horizontal scaling for handling millions of users
- Optimize read-heavy operations using caching



DEPARTMENT OF

COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

7. Learning Outcomes (What I Have Learnt)

- Understood architecture of real-time stock trading systems
- Learned importance of consistency in financial systems
- Gained knowledge of WebSocket-based real-time updates
- Understood order lifecycle (place → match → execute → settle)
- Learned how to design scalable and low-latency systems